

CAN304 Lab 6

Formal Verification of Security Protocols with ProVerif

This lab introduces ProVerif which is an automated symbolic protocol verifier for checking secrecy and authentication properties. You will learn how to use ProVerif to analyze a security protocol to identify the vulnerability, and how to fix the vulnerability and verify the fix.

Prerequisites:

- Understanding of basic cryptographic primitives (encryption, signatures, nonces)
- Familiarity with security protocol goals (confidentiality, authentication)
- Basic command-line experience

1. Introduction to ProVerif

1.1 What is ProVerif?

ProVerif is an automated symbolic protocol verifier developed by Bruno Blanchet. It operates in the Dolev-Yao model, where cryptographic primitives are treated as "black boxes" with perfect security (e.g., encryption cannot be broken without the key).

Official website: <https://bblanche.gitlabpages.inria.fr/proverif/>

1.2 How ProVerif Works

- S1. Modeling: Protocols are described as processes in an extension of the pi-calculus.
- S2. Queries: Security properties are expressed as logical queries.
- S3. Verification: ProVerif automatically determines if an attacker can violate the queries.
- S4. Results: Outputs either "verified" or provides an attack trace.

1.3 Key Concepts

- Free names: Constants like keys and nonces (key, secret)
- Functions and equations: Cryptographic operations (enc, dec, sign)
- Channels: Communication paths (c, privChan)
- Processes: Protocol participants (client, server)
- Queries: Security properties to verify (query attacker(secret))

2. Lab Environment Setup

Follow the instructions in <https://bblanche.gitlabpages.inria.fr/proverif/> to download and install ProVerif

Alternatively, you can use the online version for quick testing:

<http://proverif24.paris.inria.fr>

3. Analyze and Fix Vulnerable Protocol

3.1 Verify Woo-Lam protocol

Run ProVerif to verify the protocol, using the following command:

`proverif woo-lam.pv`

You should see (scroll up to view detailed analysis)

```
-----  
Verification summary:  
  
Query inj-event(endBparam(x,y)) ==> inj-event(beginBparam(x,y)) is false.  
  
Query inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is false.  
  
-----
```

If you are using the online version, you should:

Select “Woo-Lam shared key” -> Load Protocol -> Verify

Welcome to Online Demo for ProVerif

Proverif in OCaml by Bruno Blanchet, Vincent Cheval and Marc Sylvestre
Web interface by Sreekanth Malladi and Bruno Blanchet

Input: Select protocol or enter your protocol below:

Needham-Schroeder public key

Needham-Schroeder-Lowe

Woo-Lam shared key (variant by Gordon and Jeffrey)

Woo-Lam shared key corrected

Simpler Yahalom bidirectional

SSH Transport

Otway-Rees (strong secrecy)

naive voting protocol (guessing attack)

EKE (no guessing attack)

Augmented EKE (no guessing attack)

Signed augmented EKE (no guessing attack)

Comparison of MAC schemes

Probabilistic public-key encryption

ffgg 10

Example with [precise]

Load Protocol

Verify

You will see (scroll up to view detailed analysis)

ProVerif text output:

```
ok, secrecy assumption verified: fact unreachable attacker(Kas[])
ok, secrecy assumption verified: fact unreachable attacker(Kbs[])
Starting query inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z))
goal reachable: attacker(m_2) -> inj-event(endBfull(B[],B[],N_1),@occ27_1)
Abbreviations:
N_1 = N[xA2 = B[],kbs = Kbs[],xB = B[],!1 = @sid]
@occ27_1 = @occ27[m2 = encrypt((m5,N_1),Kbs[]),m = m_2,xA2 = B[],kbs = Kbs[],xB = B[],!1 = @sid]
RESULT inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is false.
RESULT (even event(endBfull(x,y,z)) ==> event(beginBfull(x,y,z)) is false.)
```

Verification summary:

Query inj-event(endBparam(x,y)) ==> inj-event(beginBparam(x,y)) is false.

Query inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is false.

3.2 Verify the fixed Woo-Lam protocol

Run ProVerif to verify the fixed protocol, using the following command:

[proverif woo-lam-correct.pv](#)

You should see (scroll up to view detailed analysis)

Verification summary:

Query inj-event(endBparam(x,y)) ==> inj-event(beginBparam(x,y)) is **true**.

Query inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is **true**.

If you are using the online version, you should:

Select “Woo-Lam shared key” -> Load Protocol -> Verify

Welcome to Online Demo for ProVerif

Proverif in OCaml by Bruno Blanchet, Vincent Cheval and Marc Sylvestre
Web interface by Sreekanth Malladi and Bruno Blanchet

Input: Select protocol

- Needham-Schroeder public key
- Needham-Schroeder-Lowe
- Woo-Lam shared key (variant by Gordon and Jeffrey)
- Woo-Lam shared key corrected**
- Simpler Yahalom bidirectional
- SSH Transport
- Otway-Rees (strong secrecy)
- naive voting protocol (guessing attack)
- EKE (no guessing attack)
- Augmented EKE (no guessing attack)
- Signed augmented EKE (no guessing attack)
- Comparison of MAC schemes
- Probabilistic public-key encryption
- ffgg 10
- Example with [precise]

Load Protocol

or enter your protocol below:

```
(*
WooLam shared-key auth protocol
Corrections of Gordon, Jeffrey CSFW'01

A -> B : A
B -> A : N (fresh)
A -> B : { m3, B, N }_kAS
B -> S : A,B,{ m3, B, N }_kAS
S -> B : { m5, A, N }_kBS
B: verify { m5, A, N }_kBS = the message from S

Correct.

Full agreement is wrong because the adversary can make B see something
different from { m3, B, N }_kAS (even though the server and A correctly see it)
so this is no real attack.
```

Verify

You will see (scroll up to view detailed analysis)

ProVerif text output:

```
@occ12_1 = @occ12[n = N_1,kas = Kas[],xB2 = B[],xA = A[],!1 = @sid_1]
goal reachable: b-inj-event(beginBfull(A[],A[],N_1),@occ12_1) && attacker(m_1) -> inj-event(endBfull(A[],A[],N_1),@occ27_1)
The 1st hypothesis occurs strictly before the conclusion.
The 2nd hypothesis occurs before the conclusion.
Abbreviations:
N_1 = N[xA2 = A[],kbs = Kas[],xB = A[],!1 = @sid]
@occ27_1 = @occ27[m2 = encrypt((m5,A[],N_1),Kas[]),m = m_1,xA2 = A[],kbs = Kas[],xB = A[],!1 = @sid]
@occ12_1 = @occ12[n = N_1,kas = Kas[],xB2 = A[],xA = A[],!1 = @sid_1]
RESULT inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is true.
```

Verification summary:

Query inj-event(endBparam(x,y)) ==> inj-event(beginBparam(x,y)) is true.

Query inj-event(endBfull(x,y,z)) ==> inj-event(beginBfull(x,y,z)) is true.
